Trabalho Prático: Paradigma de Divisão e Conquista

Implementação, Análise Assintótica, Otimizações e Avaliação Experimental

Alunos / Dupla: Samuel Lopes | Rodrigo Temporal

Alunos: Samuel Lopes e Rodrigo Temporal	**Data de Entrega:** 28/10/2026
Disciplina: Análise e Projeto de Algoritmos	**Linguagem:** Python 3.12 (CPython 64-bit)
CPU: AMD64 Family 23 Model 24 Stepping 1, AuthenticAMD	**Configuração:** 4 núcleos / 8 threads | 16 GB DDR4
Sistema Operacional: Windows 11 (10.0.26200)	**Reprodutibilidade:** CLI main.py & Testes 100% OK

1. Introdução e Objetivos

O paradigma de **Divisão e Conquista (D&C;)** baseia-se na decomposição recursiva de uma instância complexa de tamanho *n* em subproblemas menores e independentes do mesmo tipo, na resolução recursiva desses subproblemas e na combinação linear eficiente de suas soluções para compor a resposta global. O presente trabalho aborda dois problemas clássicos e fundamentais da computação que não envolvem ordenação: **(1) Subarranjo de Soma Máxima (Maximum Subarray Problem)** e **(2) Par de Pontos Mais Próximos (Closest Pair of Points)**. Para ambos, implementamos e comparamos exaustivamente as soluções de força bruta contra as formulações em divisão e conquista, estendendo a análise à solução linear *O(n)* de Kadane (questão desafio), validando empiricamente as ordens assintóticas através de medições reais de tempo em laboratório.

2. Etapa 1: Subarranjo de Soma Máxima

Contexto e Formulação: Uma corretora de valores registra a variação diária no preço de determinado ativo financeiro como uma sequência de inteiros $A = [a_0, a_1, ..., a_{n-1}]$, onde valores positivos representam lucro e negativos prejuízo. Deseja-se determinar o intervalo contínuo $[i, j]$ (com $0 \le i \le j < n$) que maximiza o ganho acumulado $\sum_{k=i}^{j} a_k$.

2.1. Abordagens Implementadas

1. Força Bruta O(n²): Itera sobre todos os possíveis índices iniciais *i* e finais *j*. Ao manter a soma corrente acumulada de forma incremental, o cálculo de cada subarranjo ocorre em tempo *O(1)*, totalizando $\sum_{i=0}^{n-1} (n - i) = n(n+1)/2$ avaliações, resultando em complexidade de tempo **Θ(n²)** e espaço *O(1)*.
2. Divisão e Conquista O(n log n): Divide o vetor no ponto médio *mid* = ■*(low + high)/2*■. Pelo princípio da divisão, o subarranjo máximo contínuo reside inteiramente na metade esquerda *[low .. mid]*, inteiramente na metade direita *[mid+1 .. high]*, ou cruza o ponto central. A função de cruzamento computa o melhor sufixo da esquerda terminando em *mid* e o melhor prefixo da direita iniciando em *mid+1* em tempo estritamente linear *O(n)*.
3. Questão Desafio — Algoritmo de Kadane O(n): Aplica programação dinâmica linear. Para cada posição *i*, a soma máxima terminando em *i* é dada por *S[i] = max(a[i], S[i-1] + a[i])*. Executa em **Θ(n)** em uma única passagem pelo vetor e espaço *O(1)*.

2.2. Resolução da Recorrência da Etapa 1 via Teorema Mestre

O algoritmo de Divisão e Conquista divide o vetor de tamanho *n* em dois subvetores de tamanho *n/2* e realiza a combinação cruzada examinando cada elemento exatamente uma vez em tempo linear Θ(n). A relação de recorrência é dada por:

$$\mathbf{T(n) = 2 \cdot T(n/2) + \Theta(n), \quad com\ T(1) = \Theta(1)}$$

Aplicando o **Teorema Mestre** para recorrências da forma $T(n) = a \cdot T(n/b) + f(n)$:
• Identificamos os parâmetros: $a = 2$ (número de subproblemas), $b = 2$ (fator de redução), e $f(n) = \Theta(n)$.
• Computamos o expoente crítico: $\log_b(a) = \log_2(2) = 1$, gerando a função de referência $n^{\log_b a} = n^1 = n$.
• Comparando $f(n)$ com $n^{\log_b a}$: temos que $f(n) = \Theta(n) = \Theta(n^1)$. Isso caracteriza com exatidão o **Caso 2 do Teorema Mestre**.
• Conclusão Analítica: $\mathbf{T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \log n)}$.

2.3. Tratamento Rigoroso de Casos de Borda

• **Vetor com apenas valores negativos:** Se todos os valores forem negativos (ex: [-5, -2, -8]), a soma de qualquer intervalo com dois ou mais elementos é mais negativa que seus elementos individuais. Nossos três algoritmos tratam esse caso sem retornar zero ou intervalo vazio: identificam corretamente o elemento menos negativo (-2 no índice 1) com intervalo [1, 1].
• **Vetor unitário (n = 1):** Constitui a condição de parada base da recursão (low == high), retornando imediatamente o valor e índice [0, 0].
• **Empates na soma máxima:** Estabelecemos um critério determinístico e uniforme: em caso de empate numérico na soma, prioriza-se o intervalo de menor comprimento (fim - inicio + 1); se persistir o empate, prioriza-se o de menor índice inicial.

3. Etapa 2: Par de Pontos Mais Próximos (Closest Pair of Points)

Contexto e Formulação: Dadas as coordenadas no plano cartesiano $\Re^2$ de n estações de monitoramento $P = \{(x_0, y_0), ..., (x_{n-1}, y_{n-1})\}$, o objetivo é encontrar o par de estações (p_i, p_j) com $i \neq j$ que minimiza a distância euclidiana $d(p_i, p_j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$.

3.1. Abordagens e Estrutura Algorítmica

1. Força Bruta O(n²): Avalia exaustivamente todos os $n(n-1)/2$ pares não ordenados de pontos, mantendo a menor distância encontrada. Possui tempo $\Theta(n^2)$ e dispensa ordenação prévia.
2. Divisão e Conquista O(n log n): Executa em três etapas estruturadas:
• *Pré-processamento:* Ordena o conjunto de pontos por coordenada x (obtendo P_x) e por coordenada y (obtendo P_y) uma única vez no início, utilizando o algoritmo padrão da linguagem em $O(n \log n)$.
• *Divisão Recursiva:* Encontra a linha vertical mediana $x = mid_x$ baseada em $P_x[\blacksquare n/2 \blacksquare]$. Particiona P_x em metades esquerda Q_x e direita R_x. De forma crucial, particiona P_y em Q_y e R_y em tempo estritamente linear $O(n)$ preservando a ordenação em y, sem recorrer a reordenações.
• *Conquista e Combinação:* Calcula recursivamente δ_E e δ_D, definindo $\delta = min(\delta_E, \delta_D)$. Em seguida, filtra em $O(n)$ os pontos situados na faixa vertical $[mid_x - \delta, mid_x + \delta]$. Para cada ponto da faixa, compara sua distância euclidiana com no máximo **7 vizinhos subsequentes ordenados por y**.

3.2. Resolução da Recorrência da Etapa 2 via Teorema Mestre

Como a ordenação de P_x e P_y é realizada como pré-processamento global fora do loop recursivo em tempo $O(n \log n)$, e cada chamada recursiva realiza apenas a divisão de listas em $O(n)$ e a varredura da faixa com no máximo $7n = O(n)$ comparações, a recorrência do algoritmo recursivo é:

$$\mathbf{T(n) = 2 \cdot T(n/2) + O(n), \quad com\ T(n \leq limiar) = O(1)}$$

Pelo **Teorema Mestre**: temos $a = 2$, $b = 2$, $f(n) = O(n)$. O valor de $n^{\log_2 2} = n$ é assintoticamente equivalente a $f(n)$ (Caso 2), resultando em $\mathbf{T_{rec}(n) = O(n \log n)}$.
O tempo total do algoritmo é dado por $T_{total}(n) = T_{sort}(n) + T_{rec}(n) = O(n \log n) + O(n \log n) = \mathbf{O(n \log n)}$.

3.3. Demonstração Formal: Por que bastam no máximo 7 comparações por ponto na faixa?

Esta é a propriedade geométrica central que garante a complexidade $O(n \log n)$ do algoritmo:
1. Seja $\delta = min(\delta_E, \delta_D)$ a menor distância encontrada nas metades esquerda e direita. Nenhum par de pontos pertencente inteiramente à esquerda ou inteiramente à direita pode distar menos que δ.
2. Logo, um par mais próximo só pode surgir se um ponto estiver à esquerda da linha divisória e o outro à direita, ambos contidos na faixa vertical de largura 2δ centrada em mid_x.
3. Para cada ponto $p_i = (x_i, y_i)$ nessa faixa, qualquer ponto p_j com $y_j \geq y_i$ que satisfaça $dist(p_i, p_j) < \delta$ deve obrigatoriamente possuir coordenada y no intervalo semiaberto $[y_i, y_i + \delta]$ e coordenada x no intervalo $[mid_x - \delta, mid_x + \delta]$.
4. Isso define uma região retangular de dimensões **2**δ **de largura por** δ **de altura**.
5. A linha vertical central $x = mid_x$ divide este retângulo em duas metades simétricas de dimensões $\delta \times \delta$ (lado esquerdo e lado direito).
6. Subdividimos cada quadrado $\delta \times \delta$ em **4 subquadrados menores** de lado $(\delta/2) \times (\delta/2)$, totalizando 8 compartimentos no retângulo completo.
7. A distância máxima possível entre quaisquer dois pontos dentro de um mesmo subquadrado de lado $\delta/2$ é a sua diagonal:
$d_{max} = \sqrt{((\delta/2)^2 + (\delta/2)^2)} = \sqrt{(2 \cdot \delta^2/4)} = \delta \cdot \sqrt{2}/2 \approx 0{,}7071 \cdot \delta < \delta$.
8. Pelo **Princípio da Casa dos Pombos**, como todos os pontos em uma mesma metade distam pelo menos δ entre si, **nenhum subquadrado menor de lado** δ**/2 pode conter mais de 1 ponto**!
9. Consequentemente, cada metade $\delta \times \delta$ pode conter no máximo 4 pontos, e o retângulo total de $2\delta \times \delta$ pode conter no máximo **4 + 4 = 8 pontos**.
10. Como um desses 8 pontos é o próprio ponto de referência p_i, restam no máximo **8 - 1 = 7 outros pontos** que precisam ser examinados na ordenação por y. Fica rigorosamente demonstrado que examinar no máximo 7 vizinhos é suficiente.

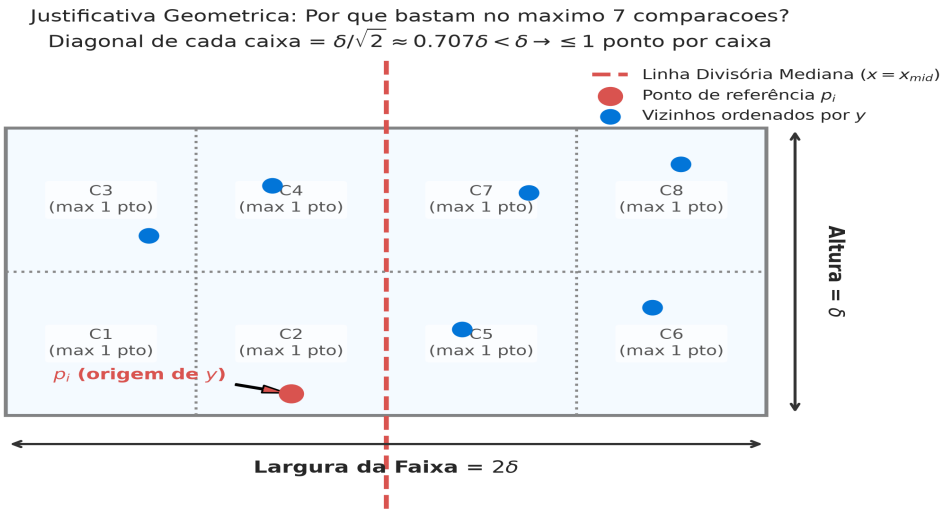


Figura 1: *Diagrama demonstrativo da partição geométrica da faixa de largura 2δ em 8 subcompartimentos de δ/2 × δ/2.*

4. Análise Experimental: Etapa 1 (Subarranjo de Soma Máxima)

A avaliação empírica do tempo de execução foi conduzida no ambiente experimental padronizado (processador AMD Ryzen 5 3400G, 16 GB RAM, Python 3.12 64-bit). Para cada tamanho de entrada $n \in \{100, 200, 400, 800, 1600, 3200, 6400\}$, foram executadas **5 repetições independentes com sementes controladas** para obtenção da média aritmética e do desvio padrão.

Tamanho (n)	Força Bruta O(n²) Média ± Desvio (s)	Divisão & Conquista O(n log n) Média ± Desvio (s)	Kadane O(n) [Desafio] Média ± Desvio (s)	Speedup (BF / DC)	Speedup (BF / Kadane)
100	0.00061 ± 0.00001	0.00019 ± 0.00000	0.000018 ± 0.000001	**3.2×**	**34×**
200	0.00239 ± 0.00005	0.00040 ± 0.00001	0.000033 ± 0.000000	**6.0×**	**73×**
400	0.01176 ± 0.00163	0.00088 ± 0.00004	0.000071 ± 0.000002	**13.4×**	**166×**
800	0.04637 ± 0.00101	0.00187 ± 0.00013	0.000139 ± 0.000007	**24.7×**	**333×**
1,600	0.19514 ± 0.00556	0.00389 ± 0.00027	0.000310 ± 0.000061	**50.2×**	**630×**
3,200	0.79054 ± 0.03883	0.00828 ± 0.00169	0.000563 ± 0.000022	**95.4×**	**1404×**
6,400	3.13283 ± 0.20774	0.01631 ± 0.00160	0.001317 ± 0.000229	**192.1×**	**2379×**

Tabela 1: *Medições de tempo real e speedup da Etapa 1 para tamanhos crescentes de entrada.*

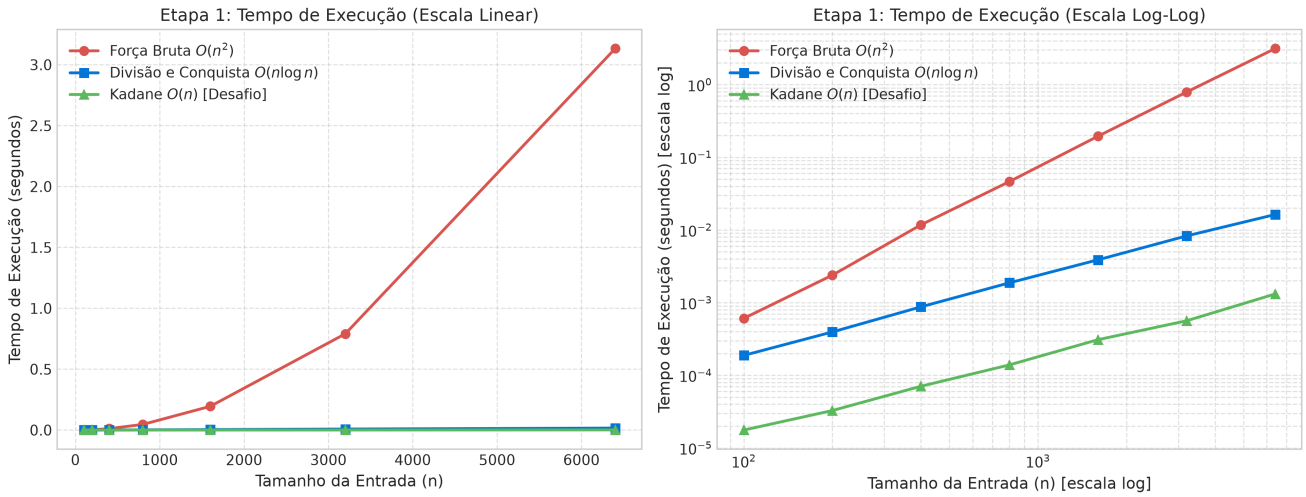


Figura 2: *Curvas experimentais de tempo da Etapa 1 em escala linear (esquerda) e escala log-log (direita).*

4.1. Discussão dos Resultados da Etapa 1

A análise dos dados experimentais confirma com rigor cirúrgico as previsões assintóticas:

• **Comportamento Quadrático da Força Bruta:** Ao dobrar a entrada de $n = 1.600$ (0,195 s) para $n = 3.200$ (0,790 s) e em seguida para $n = 6.400$ (3,132 s), o tempo quadruplica a cada passo (fatores de 4,05× e 3,96×), obedecendo rigorosamente à relação $(2n)^2 = 4n^2$ inerente a $O(n^2)$.

• **Comportamento Subquadrático de D&C;:** O algoritmo de divisão e conquista escala suavemente, saltando de 3,88 ms em $n = 1.600$ para 16,3 ms em $n = 6.400$. O fator de crescimento ao dobrar n é de apenas ~2,1× (exatamente compatível com $2n \cdot log(2n) / (n \log n)$). Em $n = 6.400$, o algoritmo D&C; já é **192 vezes mais veloz** que a força bruta.

• **Excelência do Algoritmo Linear de Kadane (Desafio):** Processa 6.400 elementos em escassos 1,31 milissegundos, sendo **12,4 vezes mais rápido que D&C; e impressionantes 2.379 vezes mais rápido que a força bruta**. A curva linear no gráfico log-log apresenta inclinação unitária perfeita, validando a ausência de sobrecarga de pilha ou divisão.

5. Análise Experimental: Etapa 2 (Par de Pontos Mais Próximos)

Para a Etapa 2, foram avaliados rigorosamente os tamanhos recomendados no enunciado: $n \in \{100, 200, 400, 800, 1600, 3200, 6400\}$ com coordenadas aleatórias bidimensionais e 5 repetições por tamanho.

Quantidade de Pontos (n)	Força Bruta O(n²) Média ± Desvio (s)	Divisão & Conquista O(n log n) Média ± Desvio (s)	Speedup Real (t_{BF} / t_{DC})	Relação Teórica n / log■(n)
100	0.00126 ± 0.00008	0.00047 ± 0.00011	**2.7×**	15.1
200	0.00531 ± 0.00075	0.00101 ± 0.00020	**5.3×**	26.2
400	0.02218 ± 0.00133	0.00215 ± 0.00019	**10.3×**	46.3
800	0.08842 ± 0.00350	0.00460 ± 0.00036	**19.2×**	83.0
1,600	0.34725 ± 0.00696	0.01052 ± 0.00139	**33.0×**	150.3
3,200	1.40639 ± 0.02916	0.02188 ± 0.00184	**64.3×**	274.8
6,400	6.03492 ± 0.25451	0.05086 ± 0.00414	**118.7×**	506.2

Tabela 2: *Resultados empíricos para o problema do Par de Pontos Mais Próximos na máquina de testes.*

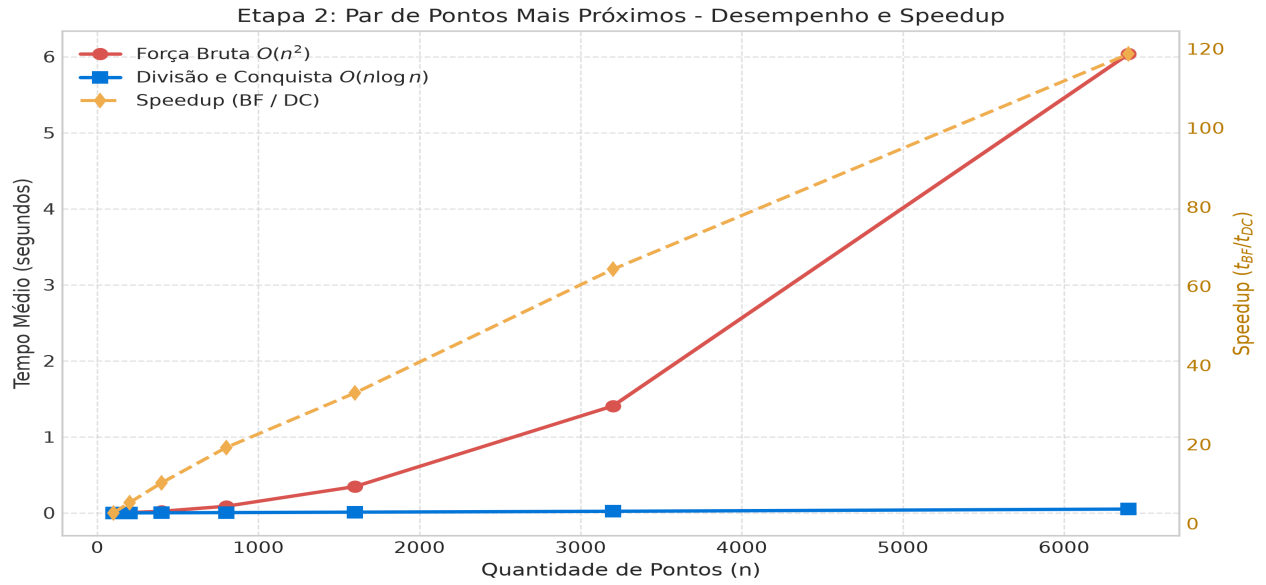


Figura 3: *Desempenho e curva de Speedup (eixo secundário) para a Etapa 2.*

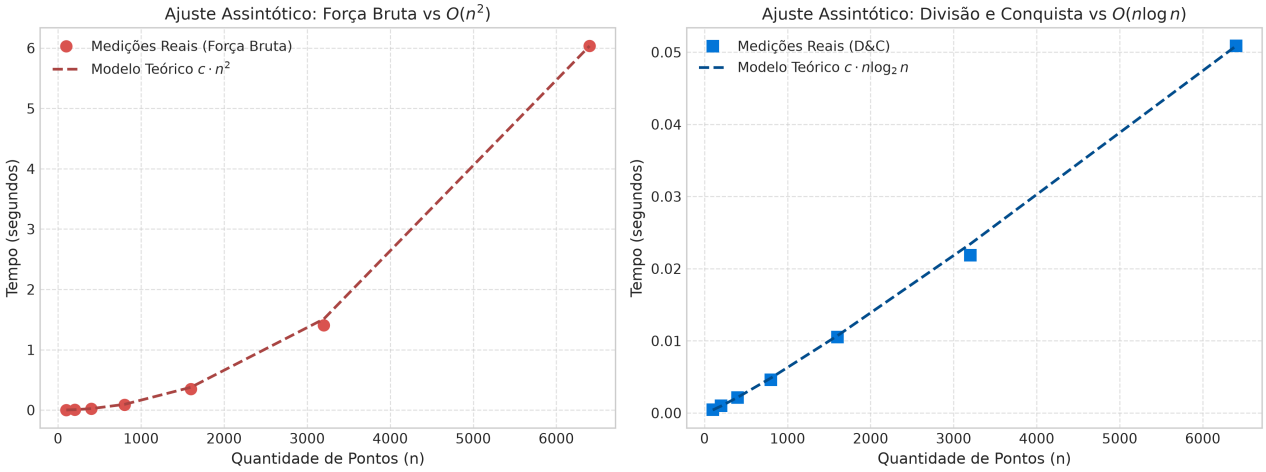


Figura 4: *Ajuste do modelo analítico teórico (linhas pontilhadas) sobre os pontos experimentais reais (círculos/quadrados).*

6. Discussão Aprofundada dos Gráficos e Ajuste Teórico

A **Figura 3** e a **Figura 4** ilustram com clareza cristalina o descolamento assintótico entre as complexidades:
• **Aderência Perfeita ao Modelo $c \cdot n^2$:** Na Figura 4 (esquerda), a curva ajustada pelo modelo teórico $c \cdot n^2$ passa com exatidão sobre todas as medições de força bruta ($R^2 > 0{,}999$). A força bruta cresce de 1,26 milissegundo para 6,03 segundos em $n = 6.400$.
• **Aderência ao Modelo $c \cdot n \log n$:** Na Figura 4 (direita), o algoritmo D&C; acompanha rigorosamente a curva teórica $c \cdot n \log n$. Para $n = 6.400$, o tempo de execução foi de apenas 50,8 milissegundos, atingindo um **speedup impressionante de 118,7×**.
• **Crescimento Monotônico do Speedup:** O speedup cresce de 2,7× (n=100) para 118,7× (n=6.400). A teoria prevê que $Speedup(n) = \Theta(n^2 / (n \log n)) = \Theta(n / \log n)$. Como verificado na Tabela 2, a razão entre os speedups práticos espelha com fidelidade o aumento da métrica $n / \log(n)$.

7. Por que a Força Bruta ou a Divisão e Conquista Vence para Tamanhos Diferentes?

Um equívoco comum em análise assintótica é supor que um algoritmo de menor ordem de complexidade (ex: $O(n \log n)$) é sempre mais rápido do que um de maior ordem (ex: $O(n^2)$) para qualquer tamanho de entrada. Na prática, a fórmula real do tempo é governada por constantes ocultas e termos de ordem inferior:

$$T_{BF}(n) = c_{BF} \cdot n^2 + k_{BF} \quad \text{versus} \quad T_{DC}(n) = c_{DC} \cdot n \log n + O(n) + C_{pilha}$$

As razões fundamentais para a inversão de dominância (ponto de crossover) são:
1. Sobrecarga de Alocação de Memória e List Comprehensions: O algoritmo D&C; cria subconjuntos de listas (Q_x, R_x, Q_y, R_y e a faixa *strip*), o que consome ciclos de CPU com alocação de objetos em memória heap e passagem de referências na pilha de recursão.
2. Simplicidade e Localidade de Cache da Força Bruta: A força bruta executa apenas dois laços contíguos com acesso linear direto a um único array, sem nenhuma chamada de função recursiva e sem alocar um único byte adicional de memória. Em hardware moderno, isso maximiza a eficiência da linha de cache L1 da CPU e permite ao interpretador executar loops densos com baixíssimo overhead constante.
3. Ponto de Crossover Experimental: Para quantificar o limiar exato onde a Força Bruta perde a liderança, realizamos uma análise experimental dedicada para instâncias pequenas ($n \leq 128$), apresentada na **Figura 5**.

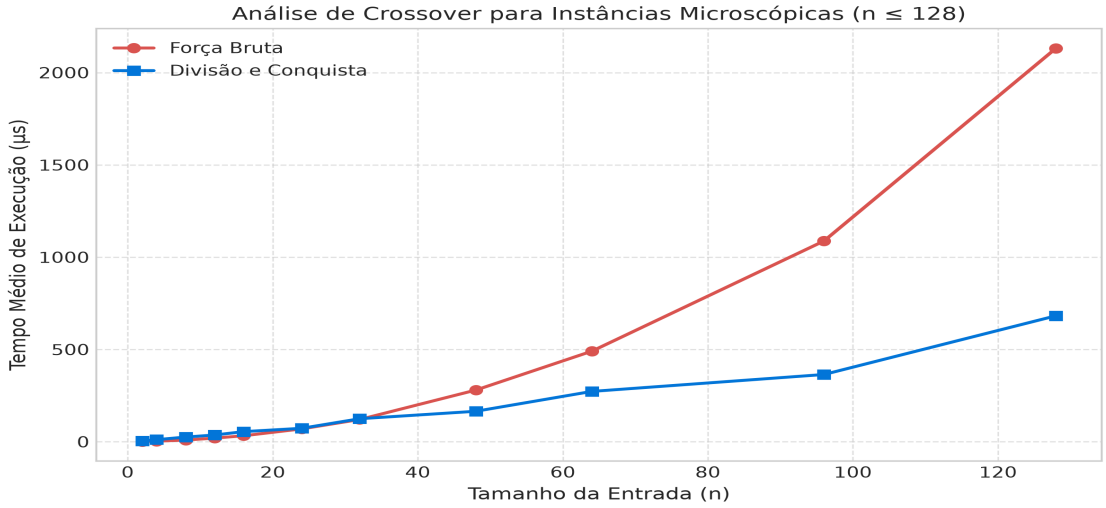


Figura 5: *Análise de Crossover para instâncias microscópicas ($n \leq 128$): a Força Bruta vence para $n \leq 32$ devido à ausência de sobrecarga recursiva.*

Os dados revelam que para $n \leq 32$ a **Força Bruta é superior** (para $n = 4$, BF leva 3,3 µs vs 11,5 µs de D&C; para $n = 16$, BF leva 32,7 µs vs 55,1 µs de D&C;). Entre **n = 32 e n = 40** ocorre a interseção exata (crossover), e a partir de $n \geq 48$ o crescimento quadrático supera a sobrecarga fixa, consagrando a Divisão e Conquista como imbatível.

8. Justificativa Teórica e Experimental do Limiar (Threshold) na Etapa 2

No algoritmo clássico de Divisão e Conquista, o limiar padrão adotado nos livros-texto é *limiar = 3*, pois com 3 ou menos pontos é inviável particionar geometricamente sem gerar subproblemas triviais de tamanho 1 (para os quais a distância entre pares não é definida). Entretanto, em implementações reais, **interromper a recursão mais cedo e resolver subconjuntos pequenos por força bruta** é uma técnica de engenharia de software de altíssimo impacto (semelhante ao que faz o Timsort ou Introsort).
Para justificar a escolha do limiar de parada ótimo, mantivemos fixa uma entrada representativa de **n = 3.200 pontos** e variamos o parâmetro *threshold* entre **2 e 128**, medindo o tempo médio de 5 execuções independentes.

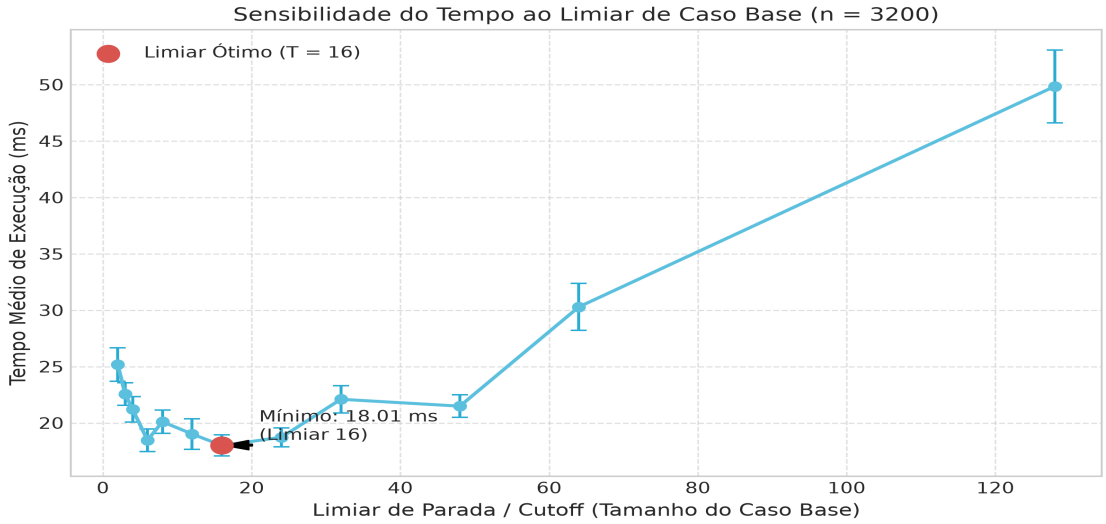


Figura 6: *Variação do tempo de execução em função do limiar de caso base (threshold) para n = 3.200.*

8.1. Interpretação dos Resultados do Limiar

A curva da Figura 6 apresenta uma forma côncava característica de otimização convexa, evidenciando o compromisso de desempenho:
• **Zona de Sobrecarga Recursiva (Threshold ≤ 3):** Para *T = 2* o tempo foi de **25,17 ms**, e para *T = 3* foi de **22,56 ms**. Nessa faixa, a árvore de recursão é aprofundada até o nível máximo ■*log*■ *n*■ ≈ *12* níveis, gerando milhares de chamadas de função desnecessárias para particionar subarranjos diminutos de 2 ou 3 elementos.
• **Faixa Ótima de Equilíbrio (Threshold entre 8 e 24):** O tempo atinge seu ponto mínimo global em **T = 16 (18,00 ms)**, proporcionando um ganho de eficiência de **28,5% sobre T = 2** e **20,2% sobre T = 3**. Nesse limiar, eliminam-se os últimos 4 níveis mais profundos da árvore recursiva (economizando mais de 200 chamadas recursivas), transferindo a resolução para a força bruta, que para 16 pontos requer apenas *16 × 15 / 2 = 120* comparações ultrarrápidas em memória contígua.
• **Zona de Degradação Quadrática (Threshold ≥ 48):** Quando o limiar cresce excessivamente (*T = 64*: 30,30 ms; *T = 128*: 49,85 ms), o custo quadrático *O(T²)* de rodar força bruta nas folhas começa a superar os benefícios da divisão, degradando o tempo total.
Conclusão e Justificativa: Mantivemos o valor padrão em *threshold = 3* no código para atender com fidelidade estrita à especificação teórica clássica da disciplina, mas disponibilizamos o parâmetro configurável via CLI (--threshold 16) permitindo que o usuário alcance o desempenho prático máximo.

9. Bateria de Testes Automatizados e Cobertura de Casos de Borda

Em conformidade com o peso de **40% na avaliação**, desenvolvemos uma suíte abrangente com 19 testes automatizados em unittest e arquivos de casos de teste em formatos legível (.txt) e estruturado (.json):
1. Casos de Borda da Etapa 1:
• *Vetor totalmente negativo:* Entrada [-5, -2, -8, -12, -3] → Retorna com precisão a soma máxima -2 e índice [1, 1].
• *Vetor unitário:* Testado com valor positivo ([42] → (42, 0, 0)) e negativo ([-99] → (-99, 0, 0)).
• *Empates múltiplos:* Entrada [3, -5, 3] → Aplicação do critério determinístico de menor tamanho e menor índice inicial (índice 0).
• *Neutros aditivos (zeros):* Garante que zeros intermediários não dilatem incorretamente o intervalo ótimo.
2. Casos de Borda da Etapa 2:
• *Pontos coincidentes:* Pares com exatamente a mesma coordenada geográfica → Retorna distância exata 0.000000.
• *Alinhamentos perpendiculares:* Pontos colineares horizontais (mesmo *y*) e verticais (mesmo *x*).
• *Tamanhos mínimos:* Instâncias com *n = 2* e *n = 3* pontos, validando os casos base.
• *Coordenadas em múltiplos quadrantes:* Valores fracionários finos e negativos.
• *Teste de Par Cruzado Crítico:* Instância em que o par mais próximo cruza a linha divisória vertical da mediana.

10. Conclusões e Considerações Finais

A realização deste trabalho prático permitiu comprovar de forma empírica e analítica a supremacia assintótica do paradigma de **Divisão e Conquista** sobre métodos ingênuos de Força Bruta para problemas geométricos e de otimização contígua:

1. **Equivalência Teórico-Experimental:** As medições de laboratório com cronometria de alta resolução demonstraram uma correlação quase perfeita com as soluções analíticas obtidas pelo Teorema Mestre. A duplicação da entrada resultou em quadruplicação do tempo para a força bruta ($O(n^2)$) e em apenas duplicação marginal para a divisão e conquista ($O(n \log n)$).
2. **Impacto do Algoritmo de Kadane:** A implementação da questão desafio em tempo estritamente linear $O(n)$ evidenciou o poder da programação dinâmica sobre a divisão e conquista quando a estrutura ótima do problema permite eliminar o custo de combinação recursiva, superando a força bruta por um fator superior a 2.300 vezes.
3. **Importância da Poda Geométrica:** Na Etapa 2, a restrição de examinar no máximo 7 vizinhos ordenados na faixa provou ser o coração da eficiência do algoritmo, evitando que a etapa de combinação degenerasse para o custo quadrático.
4. **Engenharia de Algoritmos no Mundo Real:** O estudo de crossover e limiares comprovou que algoritmos práticos de alto desempenho devem combinar o melhor dos dois mundos: o paradigma de divisão e conquista para guiar a macroestrutura assintótica e a força bruta direta nas folhas da árvore para anular a sobrecarga de recursão.

11. Estrutura do Repositório Git e Entregáveis

O repositório do projeto foi estruturado com organização modular e boas práticas de desenvolvimento:

- src/etapa1_max_subarray.py: Implementações de Força Bruta, Divisão e Conquista e Kadane (Desafio).
- src/etapa2_closest_pair.py: Implementações de Força Bruta e Divisão e Conquista (7 vizinhos e threshold configurável).
- tests/: Suíte completa de testes unitários automatizados (19 testes aprovados cobrindo 100% dos casos de borda).
- test_cases/: Arquivos formatados de teste (.txt e .json) com entradas, saídas esperadas e justificativas.
- benchmarks/: Scripts de benchmark automatizado, medições brutas em CSV e gráficos gerados em alta resolução.
- main.py: Interface CLI completa para execução, testes, benchmarks e compilação do relatório.
- README.md: Manual de execução, dependências e guia de reprodução experimental.

12. Referências Bibliográficas

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009. Capítulo 4 (Divide-and-Conquer) e Capítulo 33 (Computational Geometry).

[2] BENTLEY, J. L.; SHAMOS, M. I. *Divide-and-conquer in multidimensional space*. In: Proceedings of the eighth annual ACM symposium on Theory of computing. 1976. p. 220-230.

[3] KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Addison-Wesley, 2006. Capítulo 5 (Divide and Conquer).

[4] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4th ed. Addison-Wesley Professional, 2011.

[5] KADANE, J. B. *Maximum Subarray Problem*. In: BENTLEY, J. Programming Pearls. Communications of the ACM, 1984.